Middleware

Middleware হচ্ছে Request আর Response এর মাঝখানে দাঁড়িয়ে কাজ করা একটি processing component — তবে এটি **Server-এর ভিতরের অংশ**....

Middleware কী.....?

Middleware হলো ASP.NET Core Application-এর একটি component (class) যা client থেকে আসা HTTP Request server-এ পৌঁছানোর পর controller-এ যাওয়ার আগে execute হয়, এবং response client-এ ফেরত যাওয়ার সময়ও execute হয়....

সহজে:

Client → Server → Middleware → Controller → Middleware → Server → Client

Middleware সরাসরি client-এর আগে থাকে না।
Middleware server-এর ভিতরের request pipeline-এর অংশ।

Middleware হলো server-এর ভিতরের একটি software layer/component
যা client থেকে আসা Request এবং server থেকে ফেরত যাওয়া Response প্রসেস করে।

Middleware মানে:

- Server-এর ভিতরের মাঝখানের গেটকিপার (Gatekeeper)
- Request কে controller-এ যাওয়ার আগে প্রসেস করে
- Response ফেরত যাওয়ার আগে আবার প্রসেস করতে পারে

Real Example (ASP.NET Core)

```
app.UseAuthentication();
app.UseAuthorization();
```

এগুলো built-in middleware।
Request controller-এ যাওয়ার আগে user valid কিনা check করে।

Middleware কোথায় থাকে.....?

Middleware থাকে Request Pipeline এর মধ্যে। যখন একটি Request আসে তখন আসলে এটা সরাসরি Controller এ যায় না।বরং যায় Middleware pipeline দিয়ে:

```
Client Request
↓
Server
↓
Middleware 1
↓
Middleware 2
↓
Middleware 3
↓
Controller / Endpoint
↓
Response ফেরত আসার সময় আবার Middleware দিয়ে যায়
↓
Server
↓
Client
```

Middleware Flow Diagram

```
Client
  ↓ Request
Server
  ↓
[ Logging Middleware ]
  ↓
[ Authentication Middleware ]
  ↓
[ Authorization Middleware ]
  ↓
[ Routing Middleware ]
  ↓
Controller / API Endpoint
  ↑
Response
```

↑
[Middleware আবার কাজ করে]
↑
Server
↑
Client

Middleware কেন দরকার.....?

Middleware ব্যবহার করা হয় কারণ:

✔ Code Clean থাকে
✔ Reusable হয়
✔ Security বাড়ায়
✔ Request Processing সহজ হয়
✔ Cross-cutting Concern Handle করা যায়

Middleware কী কী কাজ করে.....?

Real World কাজগুলো দেখি:

1. Authentication Middleware

User লগইন করা আছে কিনা check করে

Example: `Request → Token` আছে? → Valid? → তারপর `API` Access

ASP.NET Core: `app.UseAuthentication();`

2. Authorization Middleware

User এর Role আছে কিনা check করে

Example: Admin `API` → শুধু Admin access করতে পারবে

`ASP.NET Core: app.UseAuthorization();`

3. Logging Middleware

সব Request এর তথ্য save করা হয়

Example:

User কোন URL এ গেল
কত সময় লাগলো
কোন IP থেকে আসলো

4. Error Handling Middleware

Server error হলে সুন্দর response পাঠায়

Example:

```
{
  "error": "Something went wrong"
}
```

ASP.NET: `app.UseExceptionHandler();`

5. CORS Middleware

Frontend (React) যেন Backend API access করতে পারে

```
app.UseCors();
```

6. Request Validation Middleware

Invalid Request আটকায়

Example:Body `empty` হলে `reject`

7. Rate Limiting Middleware

একজন user বারবার request পাঠালে block করে

Example:১ মিনিটে `100 request` এর বেশি করলে `reject`

Middleware এর বৈশিষ্ট্য

Middleware এর কিছু core বৈশিষ্ট্য:

✔ Sequential Execution

একটার পর একটা middleware execute হয়

✔ Can Stop Request

Middleware চাইলে request আটকে দিতে পারে

✔ Can Modify Request/Response

Header, Body modify করতে পারে

ASP.NET Core Middleware Example (Custom)

আমরা চাইলে নিজেও middleware বানাতে পারি:

```
public class MyMiddleware
{
    private readonly RequestDelegate _next;

    public MyMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        Console.WriteLine("Request এসেছে!");

        await _next(context);

        Console.WriteLine("Response চলে যাচ্ছে!");
    }
}
```

Pipeline এ add করতে হয়:

```
app.UseMiddleware<MyMiddleware>();
```

Middleware vs Controller

বিষয়	Middleware	Controller
কাজ	Request এর মাঝখানে প্রসেস	Main Business Logic
অবস্থান	Pipeline এ	Endpoint এ
Reusable	Global	Specific API
Example	Auth, Logging	CRUD Operations

Real Project Middleware Flow (JWT)

```
User Login
↓
Server JWT দেয়
↓
Client Token Save করে
↓
Next Request এ Token পাঠায়
↓
Middleware Token Verify করে
↓
Valid হলে Controller access
```

Middleware হলো ASP.NET Core/Express এর Request Pipeline-এর একটি component, যা client থেকে আসা HTTP request server-এ পৌঁছানোর পর middleware pipeline-এর মাধ্যমে controller বা endpoint-এ যাওয়ার আগে এবং response client-এ ফেরত যাওয়ার সময় বিভিন্ন কাজ করে, যেমন authentication, logging, error handling, routing ইত্যাদি।

ASP.NET Core Based Middleware

Middleware হলো ASP.NET Core Application-এর এমন একটি Component (Class),
যা server-এর ভিতরের Request Pipeline-এ প্রতিটি HTTP Request প্রসেস করে।

 Middleware মূলত Request এবং Response দুটোই প্রসেস করে।

Middleware সম্পর্কে গুরুত্বপূর্ণ পয়েন্টগুলো:

1) Middleware Works on HTTP Request and Response

Middleware HTTP Request ও Response এর উপর কাজ করে।

Middleware client থেকে আসা Request server-এ পৌঁছানোর পর controller-এ যাওয়ার আগে প্রসেস করে, এবং server থেকে Response client-এ ফেরত যাওয়ার সময় আবার প্রসেস করে।

2) You can configure as many middleware you want

আপনি যত খুশি Middleware Configure করতে পারেন

ASP.NET Core এ একাধিক Middleware যোগ করা যায়, যেমন:

- Authentication Middleware
- Logging Middleware
- Error Handling Middleware
- Custom Middleware

3) Order of the middleware is so important

Middleware এর Order খুবই গুরুত্বপূর্ণ

Middleware গুলো যেভাবে Pipeline এ লেখা হয়, ঠিক সেই Sequence অনুযায়ী Execute হয়।

 Wrong Order হলে Application ভুলভাবে কাজ করতে পারে।

4) If we want the next Middleware to be Processed,then we have to explicitly mention the next Middleware.

পরবর্তী Middleware Execute করতে চাইলে `next()` কল করতে হয়

যদি আমরা চাই Middleware এর পরের Middleware টি কাজ করুক, তাহলে আমাদের Explicitly Next Middleware কে Call করতে হয়।

ASP.NET Core এ: await `_next(context);`

 এটা না লিখলে Request সেখানেই থেমে যাবে।

5) You can choose to use the built in middleware or create your own middleware.

Built-in Middleware ব্যবহার করা যায় অথবা নিজের Middleware বানানো যায়

ASP.NET Core অনেক Built-in Middleware দেয় যেমন:

- `UseAuthentication()`
- `UseAuthorization()`
- `UseCors()`

এছাড়াও চাইলে আমরা Custom Middleware তৈরি করতে পারি।

Middleware হলো ASP.NET Core server-এর Request Pipeline-এর একটি Component, যা client থেকে আসা HTTP Request server-এ পৌঁছানোর পর controller-এ যাওয়ার আগে এবং Response client-এ ফেরত যাওয়ার সময় execute হয়।

এটি HTTP Pipeline-এর অংশ এবং এর execution order অত্যন্ত গুরুত্বপূর্ণ। Middleware Built-in হতে পারে অথবা Custom তৈরি করা যায়।

Middleware Related Important Topics

1) Request Pipeline Deep Understanding

Middleware শুধু concept না, এটা পুরো ASP.NET Core এর backbone।

আপনাকে বুঝতে হবে:

- Request কোথা থেকে আসে
- Middleware দিয়ে কিভাবে যায়
- Endpoint এ পৌঁছে কিভাবে Response ফেরত আসে

 Pipeline বুঝলে Middleware clear হয়ে যাবে।

2) `Use`, `Run`, `Map` Difference

এটা interview এ অনেক আসে।

`Use()`

Next middleware এ যেতে দেয়

```
app.Use(async (context, next) =>
{
    await next();
});
```

Run()

Pipeline এখানেই শেষ

```
app.Run(async context =>
{
    await context.Response.WriteAsync("End");
});
```

Map()

Specific route এর জন্য middleware

```
app.Map("/admin", adminApp =>
{
    adminApp.Use(...);
});
```

3) Built-in Middleware List (Production Must)

আপনাকে এগুলো জানা দরকার:

- `UseRouting()`
- `UseAuthentication()`
- `UseAuthorization()`
- `UseCors()`
- `UseStaticFiles()`
- `UseExceptionHandler()`

4) Custom Middleware Creation (Practical Skill)

আপনি already দেখেছেন, কিন্তু practice করা দরকার:

- Class-based Middleware
- Inline Middleware

Custom middleware লাগে:

- JWT যাচাই করতে
- Request logging করতে
- Rate limiting করতে

5) Middleware Short-Circuiting

Middleware চাইলে request থামিয়ে দিতে পারে।

Example:

```
if (!context.User.Identity.IsAuthenticated)
{
    context.Response.StatusCode = 401;
    return;
}
```

এটা security এর জন্য খুব গুরুত্বপূর্ণ।

6) Dependency Injection in Middleware

Middleware এ service inject করা যায়:

```
public class MyMiddleware
{
    private readonly RequestDelegate _next;

    public MyMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context,
ILogger<MyMiddleware> logger)
    {
        logger.LogInformation("Request এসেছে");
        await _next(context);
    }
}
```

Real project এ এটা must.

7) Exception Handling Middleware

Production app এ সবচেয়ে দরকারি।

```
app.UseExceptionHandler("/error");
```

আপনাকে জানতে হবে global error middleware কিভাবে কাজ করে।

8) Middleware Order Rules (Most Important)

Order ভুল হলে app ভেঙে যাবে।

Correct order example:

```
app.UseRouting();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();
```

Interview এ এটা খুব common প্রশ্ন।

9) Middleware vs Filters

ASP.NET Core এ confusion হয়:

Middleware	Filter
Global pipeline	Only MVC actions
Works for all requests	Works inside controller
Example: Auth, Logging	Example: Action Filter

আপনার Middleware শেখা complete হবে যখন আপনি এগুলো build করবেন:

- JWT Middleware
- Request Logging Middleware
- Rate Limiting Middleware
- Global Exception Middleware

Middleware vs Filters

ASP.NET Core এ Middleware এবং Filters দুটোই request প্রসেস করে, কিন্তু কাজ করার জায়গা (execution stage) এবং scope একদম আলাদা।

Middleware কী.....?

Middleware হলো ASP.NET Core server-এর Request Pipeline-এর অংশ,
যা client থেকে আসা HTTP request server-এ পৌঁছানোর পর controller-এ যাওয়ার আগে execute হয়, এবং response ফেরত যাওয়ার সময়ও execute হয়।

- Works globally for the whole application.

Filters কী.....?

Filters হলো ASP.NET Core MVC/Web API এর একটি feature,
যা শুধুমাত্র Controller এবং Action Method execution-এর সময় কাজ করে।

- Works inside MVC pipeline only.

Main Difference

- Middleware কাজ করে:

```
Client → Server → Middleware → Controller → Middleware → Client
```

- Filters কাজ করে:

Controller Selected

↓

Action execute হওয়ার আগে/পরে

Comparison Table

Feature	Middleware	Filters
Scope	Entire Application	Only MVC Controllers/Actions
Executes When	Request pipeline শুরুতে	Controller/Action এর সময়
Works On	All HTTP requests	Only MVC/Web API requests
Used For	Auth, Logging, CORS, Exception Handling	Validation, Action-specific logic
Access Level	Low-level HTTP context	High-level MVC context
Example	JWT Middleware	Action Filter
Runs Before Routing?	Yes	No (after routing)

Execution Flow Difference

Middleware Flow:

Client Request
↓
Server (Kestrel / IIS)
↓
Middleware 1
↓
Middleware 2
↓
Routing
↓
Controller
↓
Response
↑
Middleware গুলো আবার execute হয়
↑
Server
↑
Client

Filter Flow:

Controller Selected
↓
Authorization Filter
↓
Action Filter
↓
Action Method Execute
↓
Result Filter
↓
Response

Middleware Example

```
app.Use(async (context, next) =>
{
    Console.WriteLine("Request এসেছে");
    await next();
    Console.WriteLine("Response যাচ্ছে");
});
```

 সব request এর জন্য কাজ করবে।

Filter Example

```
public class MyActionFilter : IActionFilter
{
    public void OnActionExecuting(ActionExecutingContext
context)
    {
        Console.WriteLine("Action শুরু হওয়ার আগে");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        Console.WriteLine("Action শেষ হওয়ার পরে");
    }
}
```

 শুধু controller action এর জন্য কাজ করবে।

When to Use Middleware?

Use Middleware when:

- Authentication / JWT Verification
- Global Logging
- CORS
- Global Error Handling
- Static Files Handling

When to Use Filters?

Use Filters when:

- Controller-level validation
- Action-specific logging
- Request model validation
- Role-based access for specific actions
- Response formatting

Real Project Example

JWT Authentication → Middleware

কারণ সব API request এ token check লাগবে।

Input Validation for One Action → Filter

কারণ শুধু নির্দিষ্ট endpoint এর জন্য লাগবে।

Middleware হলো ASP.NET Core server-এর global request pipeline component, যা client থেকে আসা HTTP request server-এ পৌঁছানোর পর controller-এ যাওয়ার আগে এবং response ফেরত যাওয়ার সময় execute হয়। এটি সব HTTP request-এর জন্য application-wide কাজ করে।

আর Filters হলো ASP.NET Core MVC/Web API pipeline-এর অংশ, যা শুধুমাত্র controller এবং action execution-এর সময় কাজ করে। Filters মূলত action-specific বা controller-level logic handle করে।

Summary

Middleware হলো ASP.NET Core Application-এর একটি component/class, যা server-এর ভিতরের Request Pipeline-এ প্রতিটি HTTP Request প্রসেস করে এবং Request ও Response দুটোই handle করে।

- Middleware HTTP Request এবং Response এর উপর কাজ করে
- একাধিক Middleware configure করা যায়
- Middleware এর execution order খুব গুরুত্বপূর্ণ
- পরবর্তী Middleware চালানোর জন্য `_next()` call করতে হয়
- Built-in Middleware ব্যবহার করা যায় অথবা Custom Middleware তৈরি করা যায়

এটি ASP.NET Core Application-এর একটি গুরুত্বপূর্ণ pipeline component, যা client থেকে আসা HTTP request server-এ পৌঁছানোর পর controller-এ যাওয়ার আগে এবং server থেকে response client-এ ফেরত যাওয়ার সময় execute হয়। এটি মূলত request এবং response প্রসেস করে বিভিন্ন common কাজ যেমন authentication, authorization, logging, error handling, routing, CORS ইত্যাদি পরিচালনা করার জন্য।

ASP.NET Core-এ একাধিক middleware ব্যবহার করা যায় এবং middleware গুলো যে order-এ configure করা হয়, ঠিক সেই order অনুযায়ী pipeline-এ execute হয়। যদি কোনো middleware পরবর্তী middleware কে execute করতে চায়, তাহলে তাকে explicitly `_next()` call করতে হয়, না হলে request সেখানেই থেমে যাবে।

Middleware built-in হতে পারে অথবা প্রয়োজন অনুযায়ী custom middleware তৈরি করা যায়, যা application-কে আরো secure, scalable এবং maintainable করে তোলে।

Middleware হলো ASP.NET Core Application-এর একটি গুরুত্বপূর্ণ pipeline component, যা HTTP request server-এ আসার পর controller-এ যাওয়ার আগে এবং response client-এ ফেরত যাওয়ার সময় execute হয়। Middleware request ও response প্রসেস করে এবং authentication, authorization, logging, error handling, routing, CORS ইত্যাদি common concern handle করে।

ASP.NET Core-এ middleware গুলোর execution order অত্যন্ত গুরুত্বপূর্ণ, কারণ pipeline-এ যেভাবে configure করা হয় ঠিক সেই ক্রমেই execute হয়। `_next()` call না করলে request pipeline সেখানেই থেমে যাবে।

Thank You